MBLP131 · HAFTA 12

İşletim Sistemleri Temelleri

Hafta 12 · Metin İşleme, Yönlendirme ve Boru Hatları

Çekirdeğin birbirine bağladığı küçük programlar, gerçek veriler hakkındaki gerçek soruları yanıtlar.

Elektronik Haberleşme Teknolojisi Programı · Meslek Yüksekokulu · İstanbul Okan Üniversitesi

GEÇEN HAFTA

Dört Fikirde 11. Hafta

Kullanıcılar ve gruplar

id, whoami, groups; her kullanıcının ve grubun bir numarası (UID, GID) vardır; /etc/passwd ve /etc/group içinde listelenir.

Kip dizgesi

-rwxr-x--- : dosya türü, ardından sahip, grup ve diğerleri için rwx. Bir dizinde x, "içine girebilir" demektir.

chmod

Sembolik (u+x, go-w) ya da sekizli, r = 4, w = 2, x = 1 ile: 754 = rwx r-x r--.

Kim değiştirebilir?

umask varsayılanları belirler; chown root ister; sudo tek bir komutu root olarak çalıştırır — dikkatli kullanın.

Geçen hafta koruduğumuz dosyalar bugün veriye dönüşüyor: içeriklerini arıyor, kesiyor, sıralıyor ve sayıyoruz; programları da birbirine bağlıyoruz.

Bugünün Gündemi

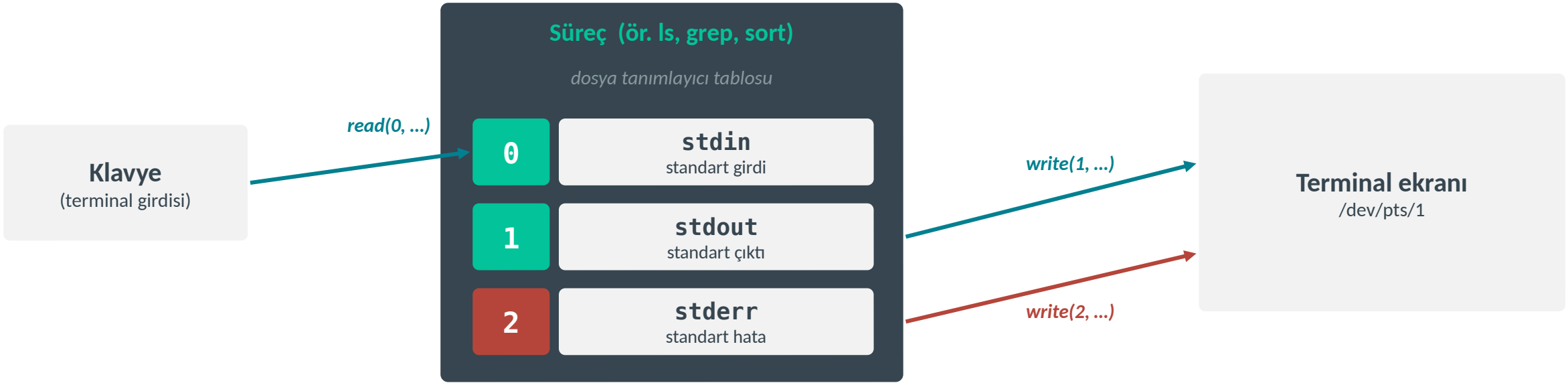
1. **Standart akışlar** stdin, stdout, stderr = 0, 1, 2 numaralı dosya tanımlayıcıları (2–3. haftalar: write(1, …)).
2. **Yönlendirme** > >> < 2> 2>&1 &> /dev/null — ve kabuğun bunu dup2() ile nasıl yaptığı.
3. **Borular** komut1 | komut2: bir çekirdek tamponuyla bağlanan iki süreç.
4. **Filtreler** grep (basit düzenli ifadelerle), sort, uniq, cut, wc, tr.
5. **Diğer araçlar** Boru hatlarında head ve tail, tee, find, xargs.
6. **Gerçek analizler + lab** Durum kodları, en çok istenen sayfalar, saat başına hatalar, notlar — her biri tek satır.

BÖLÜM 1

Standart Akışlar ve Yönlendirme

Her süreç hayata üç açık dosyayla başlar.

Üç Akış, Üç Küçük Sayı



0 · stdin

Programın girdisini okuduğu yer. Varsayılan: klavye.

1 · stdout

Normal sonuçlar. Varsayılan: terminal ekranı.

2 · stderr

Hata iletileri, ayrı bir kanalda — sonuçlar bir dosyaya gitse bile size ulaşırlar.

Bunu Daha Önce Gördünüz: write(1, …)

```
student@okan-lab:~/lab$ strace -o t1.txt -e trace=write ls data nosuch
ls: cannot access 'nosuch': No such file or directory
data:
cities.txt  numbers.txt  students.csv
student@okan-lab:~/lab$ cat t1.txt
write(2, "ls: ", 4)                      = 4
write(2, "cannot access 'nosuch'", 22)  = 22
write(2, ": No such file or directory", 27) = 27
write(2, "\n", 1)                       = 1
write(1, "data:\n", 6)                  = 6
write(1, "cities.txt  numbers.txt  student"..., 38) = 38
+++ exited with 2 +++
```

strace ne gösteriyor?

- Hata iletisi write(2, …) ile çıkar: fd 2, stderr.
- Liste write(1, …) ile çıkar: fd 1, stdout.
- ls, 1 ve 2'nin nereye gittiğini hiç sormaz: bu, ls başlamadan önce belirlenmiştir.

2. hafta: printf() sonunda write(1, "Hello…", n) olur. 3. hafta: sistem çağrısı çekirdeğe bir tuzaktır (trap) ve fd 1 standart çıktı demektir. Bugün: kabuk, ls başlamadan önce fd 1'i bir dosyaya yöneltirse her write(1, …) o dosyaya düşer. Yönlendirme budur.

Tanımlayıcıları Görmek: ls -l /proc/self/fd

```
student@okan-lab:~/lab$ ls -l /proc/self/fd
total 0
lrwx------ 1 student student 64 Oct 12 08:48 0 -> /dev/pts/1
lrwx------ 1 student student 64 Oct 12 08:48 1 -> /dev/pts/1
lrwx------ 1 student student 64 Oct 12 08:48 2 -> /dev/pts/1
lr-x------ 1 student student 64 Oct 12 08:48 3 -> /proc/15829/fd
student@okan-lab:~/lab$ ls -l /proc/self/fd > fd.txt
student@okan-lab:~/lab$ cat fd.txt
total 0
lrwx------ 1 student student 64 Oct 12 08:48 0 -> /dev/pts/1
l-wx------ 1 student student 64 Oct 12 08:48 1 -> /home/student/lab/fd.txt
lrwx------ 1 student student 64 Oct 12 08:48 2 -> /dev/pts/1
lr-x------ 1 student student 64 Oct 12 08:48 3 -> /proc/15832/fd
student@okan-lab:~/lab$ ls -l /proc/self/fd 2> /dev/null
total 0
lrwx------ 1 student student 64 Oct 12 08:48 0 -> /dev/pts/1
lrwx------ 1 student student 64 Oct 12 08:48 1 -> /dev/pts/1
l-wx------ 1 student student 64 Oct 12 08:48 2 -> /dev/null
lr-x------ 1 student student 64 Oct 12 08:48 3 -> /proc/15841/fd
```

Neye bakmalı?

- /proc/self, bakan süreçtir — burada ls'in kendisi (3. hafta: /proc).
- Normalde 0, 1, 2'nin üçü de /dev/pts/1 terminalini gösterir.
- > fd.txt ile fd 1 dosyayı gösterir: ls kendi listesini dosyaya yazdı.
- 2> /dev/null ile fd 2 "kara delik" aygıtını gösterir.
- fd 3, ls'in okuduğu dizindir.

Çıktı Yönlendirme: > Üzerine Yazar, >> Sona Ekler

```
student@okan-lab:~/lab$ echo "first line" > notes.txt
student@okan-lab:~/lab$ cat notes.txt
first line
student@okan-lab:~/lab$ echo "second line" >> notes.txt
student@okan-lab:~/lab$ cat notes.txt
first line
second line
student@okan-lab:~/lab$ echo "oops" > notes.txt
student@okan-lab:~/lab$ cat notes.txt
oops
```

Kurallar

- **komut > dosya** dosyayı oluşturur ya da önce boşaltır (truncate), sonra stdout'u içine yazar.
- **komut >> dosya** gerekirse oluşturur, sonra sonuna ekler.
- Ekranda hiçbir şey görünmez: stdout dosyaya gitti.

Dikkat

> dosyayı komut daha başlamadan boşaltır. Tek bir yanlış > bir dosyayı yok eder — geri alma da çöp kutusu da yoktur (10. hafta).

Girdi Yönlendirme: < Bir Dosyayı stdin'e Verir

```
student@okan-lab:~/lab$ wc -l logs/access.log
300 logs/access.log
student@okan-lab:~/lab$ wc -l < logs/access.log
300
student@okan-lab:~/lab$ ls -l /proc/self/fd < docs/syllabus.txt
total 0
lr-x------ 1 student student 64 Oct 12 08:48 0 ->
/home/student/lab/docs/syllabus.txt
lrwx------ 1 student student 64 Oct 12 08:48 1 -> /dev/pts/1
lrwx------ 1 student student 64 Oct 12 08:48 2 -> /dev/pts/1
lr-x------ 1 student student 64 Oct 12 08:48 3 -> /proc/15838/fd
```

Bir dosyayı okumanın iki yolu

- **wc -l dosya:** wc dosyayı kendisi açar ve adını bilir, bu yüzden adı yazar.
- **wc -l < dosya:** dosyayı kabuk fd 0 olarak açar; wc yalnızca bir akış görür, ad yok.

Metin araçlarının çoğu filtre (filter) geleneğine uyar: argüman olarak verilen dosyaları, hiç dosya yoksa stdin'i okur; sonucu stdout'a yazar. Borularla zincirlenebilmelerinin nedeni budur — bugünün ana konusu.

Hataları Yönlendirmek: 2>, 2>&1, &> ve /dev/null

```
student@okan-lab:~/lab$ ls data nosuch > out.txt
ls: cannot access 'nosuch': No such file or directory
student@okan-lab:~/lab$ ls data nosuch 2> err.txt
data:
cities.txt  numbers.txt  students.csv
student@okan-lab:~/lab$ cat err.txt
ls: cannot access 'nosuch': No such file or directory
student@okan-lab:~/lab$ ls data nosuch > all.txt 2>&1
student@okan-lab:~/lab$ cat all.txt
ls: cannot access 'nosuch': No such file or directory
data:
cities.txt
numbers.txt
students.csv
student@okan-lab:~/lab$ ls data nosuch 2> /dev/null
data:
cities.txt  numbers.txt  students.csv
```

Sözdizimi	Anlamı
`> f  ya da  1> f`	stdout → f
`2> f`	stderr → f
`2>> f`	stderr'i f'nin sonuna ekle
`> f 2>&1`	ikisi de f'ye (fd 2 = fd 1'in kopyası)
`&> f`	ikisi de f'ye (bash kısaltması)
`2> /dev/null`	hataları at

/dev/null bir aygıt dosyasıdır: ona yazılan her şey kaybolur; ondan okumak hemen dosya sonu verir.

Sıra Önemlidir: Yönlendirmeler Soldan Sağa İşlenir

`ls data nosuch > all.txt 2>&1`

başlangıç	fd 1 → terminal	fd 2 → terminal
> all.txt	**fd 1 → all.txt**	fd 2 → terminal
2>&1	**fd 1 → all.txt**	**fd 2 → all.txt**

İki akış da all.txt dosyasına gider.

`ls data nosuch 2>&1 > out2.txt`

başlangıç	fd 1 → terminal	fd 2 → terminal
2>&1	fd 1 → terminal	fd 2 → terminal
> out2.txt	**fd 1 → out2.txt**	fd 2 → terminal

Hata yine ekranda görünür!

```
student@okan-lab:~/lab$ ls data nosuch 2>&1 > out2.txt
ls: cannot access 'nosuch': No such file or directory
# 2>&1 şu demektir: fd 2'yi fd 1'in ŞU ANKİ hâlinin kopyası yap — fd 1'e kalıcı bir bağ değil
```

Kabuk Bunu Nasıl Yapar: fork, open, dup2, exec

```
student@okan-lab:~/lab$ strace -f -o t2.txt -e trace=openat,dup2,execve bash -c "ls data > list.txt"
student@okan-lab:~/lab$ grep -E "list.txt|dup2|execve" t2.txt
15726 execve("/usr/bin/bash", ["bash", "-c", "ls data > list.txt"], …) = 0
15727 openat(AT_FDCWD, "list.txt", O_WRONLY|O_CREAT|O_TRUNC, 0666) = 3
15727 dup2(3, 1)                        = 1
15727 execve("/usr/bin/ls", ["ls", "data"], …) = 0
```

1 fork (clone)

bash bir çocuk oluşturur: aynı 0, 1, 2 tanımlayıcılarına sahip bir kopyası.

2 openat

Çocuk list.txt'yi yazmak için açar (O_TRUNC = boşalt). Yeni fd: 3.

3 dup2(3, 1)

fd 1, fd 3'ün kopyası olur: stdout artık list.txt'ye gider.

4 execve ls

Çocuk ls'e dönüşür. fd tablosu exec'ten sağ çıkar — ls fd 1'e yazar.

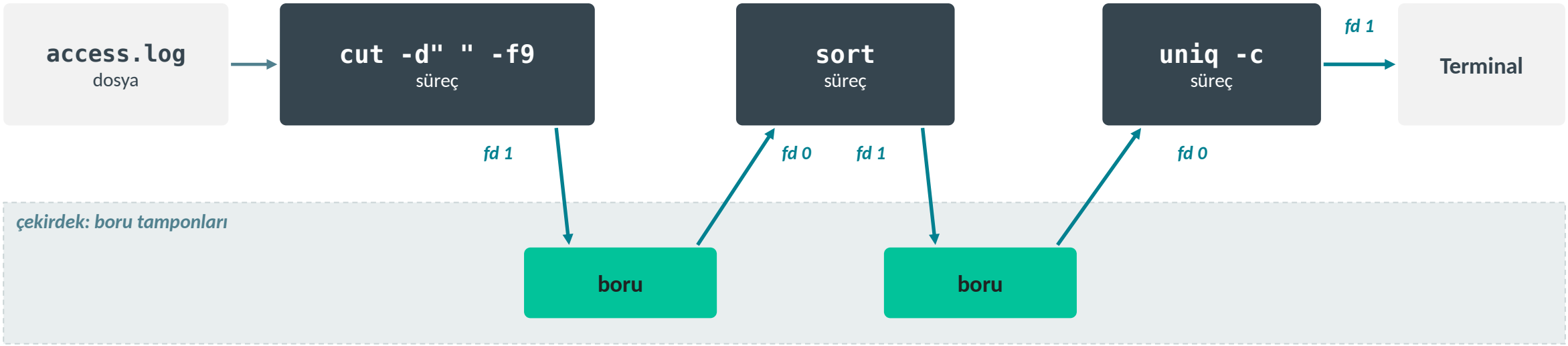
3. hafta sordu: fork() ve exec() neden iki ayrı çağrı? Çünkü ikisinin arasında çocuk kendi dosya tanımlayıcılarını yeniden düzenleyebilir — ve sonra çalıştırdığı program bunu hiç bilmez. Linux fork() çağrısını clone() sistem çağrısıyla gerçekler.

BÖLÜM 2

Borular

Bir sürecin çıktısı bir sonrakinin girdisi olur.

Boru Hattı: Borularla Bağlanmış Süreçler




```
student@okan-lab:~/lab$ cut -d" " -f9 logs/access.log | sort | uniq -c
    148 200
     20 301
     34 304
     17 403
     57 404
     24 500
```

- Üç süreç aynı anda çalışır; her boru çekirdek içinde bir tampondur.
- cut fd 1'e = boruya yazar; sort fd 0'dan = borudan okur.
- Her aşamanın stderr'i (fd 2) yine terminale gider.

Boru Kurmak: pipe2, clone, dup2, execve

```
student@okan-lab:~/lab$ strace -f -o t3.txt -e trace=pipe2,clone,dup2,execve bash
-c "grep ERROR logs/system.log | wc -l"
13
student@okan-lab:~/lab$ cat t3.txt
...
15736 pipe2([3, 4], 0) = 0
15736 clone(…) = 15737
15736 clone(…) = 15738
15737 dup2(4, 1) = 1
15738 dup2(3, 0) = 0
15738 execve("/usr/bin/wc", ["wc", "-l"], …) = 0
15737 execve("/usr/bin/grep", ["grep", "ERROR", "logs/system.log"], …) = 0
...
```

İzi okumak

- **pipe2([3, 4]):** kabuk bir boru alır: fd 3 = okuma ucu, fd 4 = yazma ucu.
- **clone × 2:** iki çocuk (fork).
- **grep çocuğu:** dup2(4, 1) — stdout boruya.
- **wc çocuğu:** dup2(3, 0) — stdin borudan.

```
student@okan-lab:~/lab$ ls -l /proc/self/fd | cat
l-wx------ 1 student student 64 Oct 12 08:48 1 -> pipe:[29942]
# ls'in fd 1'i artık terminal değil: bir boru (çekirdekteki numarası 29942)
```

Boru, Çekirdeğin Yönettiği Sınırlı Bir Tampondur



boru tamponu: Linux'ta 64 KiB (16 sayfa × 4 KiB)

- **Tampon dolu:** write() bloke olur — yazan bekler.
- **Tampon boş:** read() bloke olur — okuyan bekler.
- **Yazan kapattı ve tampon boş:** read() 0 döndürür = dosya sonu.

5. hafta

Üretici–tüketici problemi — sizin yerinize çekirdek çözüyor.

```
student@okan-lab:~/lab$ time (sleep 2 | sleep 2)
real    0m2.005s
user    0m0.003s
sys     0m0.002s
student@okan-lab:~/lab$ yes | head -n 3
y
y
y
```

BÖLÜM 3

Filtreler: grep, sort, uniq, cut, wc, tr

Her biri dosyalardan ya da stdin'den satır okur, stdout'a satır yazar.

grep: Bir Kalıpla Eşleşen Satırları Yazdırır

```
student@okan-lab:~/lab$ grep ERROR logs/system.log | head -n 3
2026-10-12 09:12:00 ERROR failed to mount /dev/sdb1
2026-10-12 09:56:00 ERROR failed to mount /dev/sdb1
2026-10-12 09:07:00 ERROR failed to mount /dev/sdb1
student@okan-lab:~/lab$ grep -c ERROR logs/system.log
13
student@okan-lab:~/lab$ grep -i "TALK" docs/linux_quotes.txt
Talk is cheap. Show me the code.
student@okan-lab:~/lab$ grep -n "backup" logs/system.log | head -n 4
6:2026-10-12 09:55:00 INFO  backup finished
7:2026-10-12 09:06:00 INFO  backup started
10:2026-10-12 09:39:00 INFO  backup finished
29:2026-10-12 10:08:00 INFO  backup started
student@okan-lab:~/lab$ grep -v INFO logs/system.log | head -n 4
2026-10-12 09:11:00 WARN  high memory usage
2026-10-12 09:22:00 WARN  high memory usage
2026-10-12 09:12:00 ERROR failed to mount /dev/sdb1
2026-10-12 09:23:00 WARN  temperature 71C
student@okan-lab:~/lab$ grep -vc INFO logs/system.log
41
```

Seçenek	Anlamı
`-i`	büyük/küçük harf ayrımı yok
`-n`	satır numaralarını göster
`-v`	tersine çevir: eşleşMEYEN satırlar
`-c`	eşleşen satırları say
`-w`	yalnızca tam sözcük eşleşmesi
`-r`	dizinlerde özyinelemeli ara

Biçim: grep [seçenekler] KALIP [dosyalar]. Boşluk ya da özel karakter içeren kalıbı tırnak içine alın.

Tam Sözcükler (-w) ve Tüm Dizinler (-r)

```
student@okan-lab:~/lab$ grep "Week 1" docs/syllabus.txt
Week 1  Introduction and history
Week 10 File and directory operations
Week 11 Permissions, ownership and users
Week 12 Text processing and redirection
Week 13 Processes and job control
Week 14 Introduction to shell scripting
Week 15 Final exam
student@okan-lab:~/lab$ grep -w "Week 1" docs/syllabus.txt
Week 1  Introduction and history
student@okan-lab:~/lab$ grep -r "Project" projects
projects/beta/README.md:# Project beta
projects/alpha/README.md:# Project alpha
```

-w

"Week 1", "Week 10" … "Week 15" içinde de eşleşir. -w ile eşleşme tam bir sözcük olmalıdır: önünde ya da arkasında harf, rakam veya _ olamaz.

-r

Bir dizinin altındaki her dosyada arar ve her satırın önüne dosyanın adını yazar. Sıra alfabetik değil, dizindeki sıradır.

Basit Düzenli İfadeler

Simge	Eşleştiği	Örnek
`^`	satırın başı	`^Week`
`$`	satırın sonu	`s$`
`.`	herhangi TEK bir karakter	`c.de → code`
`[ ]`	bir kümeden tek karakter	`[0-9]  [aeiou]`
`*`	önündeki öğe, 0 ya da daha çok kez	`[0-9]*`
`\.`	gerçek bir nokta	`5\.38`

Kalıbı her zaman tırnak içine alın: yoksa kabuk, grep görmeden önce * ve [ ] karakterlerini dosya adı joker karakterleri olarak genişletebilir (10. hafta).

```
student@okan-lab:~/lab$ grep "^Week [0-9] " docs/syllabus.txt
Week 1  Introduction and history
Week 2  Definitions, goals, components, structures
Week 3  OS services, system calls, processes
...
student@okan-lab:~/lab$ grep "s$" docs/syllabus.txt
Week 2  Definitions, goals, components, structures
Week 3  OS services, system calls, processes
Week 4  Threads and CPU scheduling basics
Week 7  Virtual memory and file systems
Week 10 File and directory operations
Week 11 Permissions, ownership and users
student@okan-lab:~/lab$ grep "c.de" docs/linux_quotes.txt
Talk is cheap. Show me the code.
student@okan-lab:~/lab$ grep -c "5.38" logs/access.log
39
student@okan-lab:~/lab$ grep -c "5\.38" logs/access.log
37
student@okan-lab:~/lab$ grep ",ECT,[0-9]*,9[0-9]$" data/students.csv
2026001,Elif Yilmaz,ECT,96,99
2026015,Deniz Celik,ECT,95,96
```

sort: Alfabetik, Sayısal, Sütuna Göre

```
student@okan-lab:~/lab$ sort data/cities.txt | head -n 4
Ankara
Ankara
Antalya
Antalya
student@okan-lab:~/lab$ tail -n +2 data/students.csv | cut -d, -f5 | sort | head -n 3
100
35
36
student@okan-lab:~/lab$ tail -n +2 data/students.csv | cut -d, -f5 | sort -n | head -n
3
35
36
36
student@okan-lab:~/lab$ sort -t, -k5 -nr data/students.csv | head -n 3
2026010,Onur Dogan,ME,73,100
2026001,Elif Yilmaz,ECT,96,99
2026020,Emre Dogan,ME,64,98
```

Seçenekler

- **-n** sayısal: 35 < 100 (düz sort karakterleri karşılaştırır: '1' < '3' olduğundan "100" < "35").
- **-r** ters sıra (en büyük önce).
- **-t,** alan ayırıcı virgüldür.
- **-k5** 5. alana göre sırala (burada: final notu).
- **-u** benzersiz: tekrarlanan satırları at.

uniq: Bitişik Tekrarları Birleştirir — Önce Sıralayın!

```
student@okan-lab:~/lab$ uniq -c data/cities.txt | head -n 4
      1 Antalya
      1 Trabzon
      1 Konya
      1 Bursa
student@okan-lab:~/lab$ sort data/cities.txt | uniq -c
      2 Ankara
      6 Antalya
      5 Bursa
      5 Eskisehir
      1 Istanbul
      2 Izmir
      4 Konya
      5 Trabzon
student@okan-lab:~/lab$ sort data/cities.txt | uniq -c | sort
-rn | head -n 3
      6 Antalya
      5 Trabzon
      5 Eskisehir
```

```
student@okan-lab:~/lab$ cut -d, -f2 data/students.csv | sort |
uniq -d
Elif Yilmaz
Mehmet Aydin
Onur Dogan
Selin Yildiz
```

Unutmayın

- uniq bir satırı yalnızca hemen önceki satırla karşılaştırır.
- **-c** her grubu say;
- **-d** yalnızca tekrarlanan satırları yaz (burada: iki kez geçen adlar).
- **sort | uniq -c | sort -rn** = "her birinden kaç tane, en sık olan önce" — günün en yararlı boru hattı.

cut Sütun Alır, wc Sayar

```
student@okan-lab:~/lab$ head -n 3 data/students.csv | cut -d, -f2,5
name,final
Elif Yilmaz,99
Zeynep Dogan,36
student@okan-lab:~/lab$ head -n 2 logs/access.log | cut -d" " -f1,7,9
10.0.1.10 /about.html 404
10.0.1.10 /api/grades 200
student@okan-lab:~/lab$ head -n 3 logs/system.log | cut -c12-16
09:00
09:11
09:22
student@okan-lab:~/lab$ wc logs/system.log
 120  669 5207 logs/system.log
student@okan-lab:~/lab$ wc -l data/*
  30 data/cities.txt
  20 data/numbers.txt
  41 data/students.csv
  91 total
```

cut

- **-d',' -f2,5** virgülle ayrılmış 2. ve 5. alanlar.
- **-d" " -f7** tek boşlukla ayrılmış 7. alan.
- **-c12-16** 12–16. karakterler (sabit sütunlar).

wc

Satır, sözcük ve bayt sayısını yazar: 120 satır, 669 sözcük, 5207 bayt. Yalnızca biri için -l, -w, -c.

tr: Karakterleri Dönüştür, Sil ve Sıkıştır

```
student@okan-lab:~/lab$ head -n 2 docs/linux_quotes.txt | tr a-z A-Z
TALK IS CHEAP. SHOW ME THE CODE.
GIVEN ENOUGH EYEBALLS, ALL BUGS ARE SHALLOW.
student@okan-lab:~/lab$ head -n 2 data/students.csv | tr "," ";"
id;name;department;midterm;final
2026001;Elif Yilmaz;ECT;96;99
student@okan-lab:~/lab$ echo "Hello, World!" | tr -d ",!"
Hello World
student@okan-lab:~/lab$ head -n 3 logs/system.log | cut -d" " -f4



student@okan-lab:~/lab$ head -n 3 logs/system.log | tr -s " " | cut -d" " -f4-
network link up
high memory usage
high memory usage
```

Üç iş

- **tr a-z A-Z** 1. kümenin her karakterini 2. kümedeki karşılığıyla değiştirir.
- **tr -d küme** bu karakterleri siler.
- **tr -s " "** tekrarları teke indirir.

Boş satırlar neden?

"INFO network" içinde İKİ boşluk var, bu yüzden cut boş bir 4. alan görür. tr -s " " önce onları sıkıştırır; -f4- 4. alandan sona kadar demektir.

Bir Bakışta Yönlendirme ve Filtreler

Ne yapmak istiyorsunuz?	Kullanın	Bugünden örnek
çıktıyı kaydet / ekle	> f >> f	ls data > list.txt
girdiyi dosyadan oku	< f	wc -l < logs/access.log
hataları kaydet ya da gizle	2> f 2> /dev/null	find . -name "*.md" 2> /dev/null
her şeyi tek dosyaya	> f 2>&1 &> f	ls data nosuch > all.txt 2>&1
iki programı bağla	kmt1 | kmt2	grep ERROR logs/system.log | wc -l
eşleşen satırları bul	grep -i -n -v -c -w -r	grep -c " 404 " logs/access.log
satırları sırala	sort -n -r -t, -k	sort -t, -k5 -nr data/students.csv
her değeri say	sort | uniq -c	sort data/cities.txt | uniq -c
sütun al	cut -d -f / -c	cut -d, -f2,5 data/students.csv
karakter değiştir	tr tr -d tr -s	tr a-z A-Z < docs/linux_quotes.txt

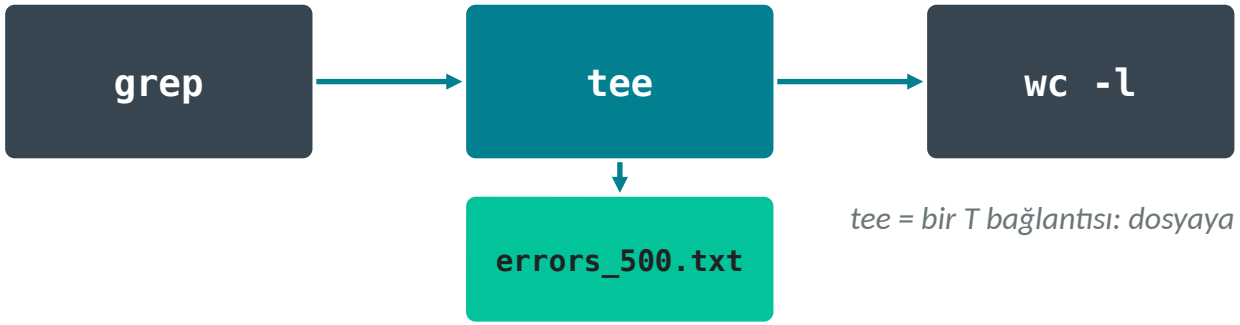
ARA

10 Dakika Mola

Sırada: head, tail, tee, find, xargs — ardından gerçek sorular, tek satırlık boru hatlarıyla.

Boru Hatlarında head, tail ve tee

```
student@okan-lab:~/lab$ tail -n +2 data/students.csv | head -n 2
2026001,Elif Yilmaz,ECT,96,99
2026002,Zeynep Dogan,ME,95,36
student@okan-lab:~/lab$ head -n 120 logs/access.log | tail -n 1
10.0.3.24 - - [12/Oct/2026:12:53:47 +0300] "GET /index.html HTTP/1.1" 404 4270
student@okan-lab:~/lab$ grep " 500 " logs/access.log | tee errors_500.txt | wc -l
24
student@okan-lab:~/lab$ wc -l errors_500.txt
24 errors_500.txt
```



tee = bir T bağlantısı: dosyaya kopyalar VE iletir

Yararlı biçimler

- **head -n 5** ilk 5 satır.
- **tail -n 5** son 5 satır.
- **tail -n +2** 2. satırdan itibaren: başlığı atlar.
- **head -n 120 | tail -n 1** tam olarak 120. satır.
- **tail -f log** büyüyen bir dosyayı izle (durdurmak için Ctrl+C).
- **tee -a f** üzerine yazmak yerine sona ekle.

find: Dizin Ağacında Ad, Tür, Boyut ve Yaşa Göre Arama

```
student@okan-lab:~/lab$ find docs -name "*.txt"
docs/notes_02.txt
docs/syllabus.txt
docs/notes_01.txt
docs/notes_03.txt
docs/linux_quotes.txt
docs/final report.txt
student@okan-lab:~/lab$ find . -type d -name src
./src
./projects/beta/src
./projects/alpha/src
student@okan-lab:~/lab$ find . -size +10k
./logs/access.log
student@okan-lab:~/lab$ find photos -iname "*.jp*g" | wc -l
14
student@okan-lab:~/lab$ touch -d "2026-09-01" docs/notes_01.txt
student@okan-lab:~/lab$ find docs -mtime +30
docs/notes_01.txt
```

Sınama	Anlamı
`-name "*.txt"`	ad eşleşir (joker karakterler tırnakta)
`-iname`	aynısı, harf büyüklüğüne bakmadan
`-type f / d`	normal dosya / dizin
`-size +10k`	10 KiB'tan büyük (-10k: küçük)
`-mtime +30`	30 günden daha önce değiştirilmiş
`-mtime -1`	son 24 saatte değiştirilmiş

Biçim: find NEREDE SINAMALAR. Birden çok sınama = hepsi sağlanmalı (VE). ls'in aksine find, NEREDE'nin altındaki bütün ağacı dolaşır.

find'ın Bulduklarıyla İş Yapmak: -exec ve xargs

```
student@okan-lab:~/lab$ find . -name "*.c" -exec wc -l {} \;
7 ./src/hello.c
1 ./projects/beta/src/main.c
1 ./projects/alpha/src/main.c
student@okan-lab:~/lab$ find . -name "*.c" | xargs wc -l
  7 ./src/hello.c
  1 ./projects/beta/src/main.c
  1 ./projects/alpha/src/main.c
  9 total
student@okan-lab:~/lab$ find docs -name "*.txt" | xargs wc -l
  1 docs/notes_02.txt
 15 docs/syllabus.txt
  1 docs/notes_01.txt
  1 docs/notes_03.txt
  5 docs/linux_quotes.txt
wc: docs/final: No such file or directory
wc: report.txt: No such file or directory
 23 total
student@okan-lab:~/lab$ find docs -name "*.txt" -print0 | xargs -0 wc -l
  1 docs/notes_02.txt
 15 docs/syllabus.txt
  1 docs/notes_01.txt
  1 docs/notes_03.txt
  5 docs/linux_quotes.txt
  2 docs/final report.txt
 25 total
```

İki yol

- **-exec kmt {} \;** kmt'yi her dosya için bir kez çalıştırır; {} = dosya adı.
- **| xargs kmt** stdin satırlarını ARGÜMANLARA çevirir: tüm adlarla tek bir wc (toplama bakın).
- **-print0 | xargs -0** boşluklu adlar için güvenli.

Güvenlik

find … -exec rm {} \; ya da … | xargs rm demeden önce aynı satırı rm OLMADAN çalıştırıp listeyi okuyun. rm geri alınamaz.

BÖLÜM 5

Gerçek Veri, Tek Satırlık Yanıtlar

logs/access.log · logs/system.log · data/students.csv

Hangi Alan Nerede?

logs/access.log — cut -d" " -fN

-f1	-f2	-f3	-f4	-f5	-f6	-f7	-f8	-f9	-f10
10.0.1.10	-	-	[12/Oct/2026:08:00:00	+0300]	"GET	/about.html	HTTP/1.1"	404	442
IP			*tarih ve saat*	*dilim*	*yöntem*	*sayfa*	*protokol*	*durum*	*bayt*

logs/system.log — sabit sütunlar: cut -cA-B

-c1-10	-c12-19	-c21-25	-c27-
2026-10-12	09:11:00	WARN	high memory usage
tarih	*saat (saat: 12-13)*	*düzey*	*ileti*

data/students.csv — cut -d, -fN (1. satır başlıktır!)

-f1	-f2	-f3	-f4	-f5
2026001	Elif Yilmaz	ECT	96	99
numara	*ad*	*bölüm*	*vize*	*final*

Her boru hattından önce

- Dosyaya head -n 3 ile bakın.
- Ayırıcıyı bulun: boşluk, virgül, sabit sütunlar?
- Alanları 1'den sayın.

YÖNTEM

Boru Hattını Her Seferinde Bir Aşama Ekleyerek Kurun

```
student@okan-lab:~/lab$ cut -d" " -f9 logs/access.log | head -n 3
404
200
500
student@okan-lab:~/lab$ cut -d" " -f9 logs/access.log | sort | head -n 3
200
200
200
student@okan-lab:~/lab$ cut -d" " -f9 logs/access.log | sort | uniq -c | sort -rn
    148 200
     57 404
     34 304
     24 500
     20 301
     17 403
```

1

Çıkar

gereken sütunu cut ile alın; head ile denetleyin.

2

Grupla

sort eşit değerleri bir araya getirir.

3

Say

uniq -c her grubu sayar.

4

Sırala

sort -rn: en büyük sayı önce; ilk N için head -n N.

En Çok İstenen Sayfalar, En Sık Ziyaretçiler, Kırık Bağlantılar

```
student@okan-lab:~/lab$ cut -d" " -f7 logs/access.log | sort | uniq -c | sort -rn | head -n 5
     41 /courses/mcmp117
     37 /about.html
     33 /index.html
     33 /images/logo.png
     32 /api/grades
student@okan-lab:~/lab$ cut -d" " -f1 logs/access.log | sort | uniq -c | sort -rn | head -n 3
     58 10.0.2.17
     56 10.0.4.31
     52 10.0.6.45
student@okan-lab:~/lab$ grep " 404 " logs/access.log | cut -d" " -f7 | sort | uniq -c | sort -rn | head -n 3
      9 /images/logo.png
      7 /index.html
      7 /about.html
```

Saat Başına Hatalar, Uyarılar, Bölümler, En İyi Finaller

```
student@okan-lab:~/lab$ grep -c ERROR logs/system.log
13
student@okan-lab:~/lab$ grep ERROR logs/system.log | cut -c12-13 | uniq -c
      3 09
      4 10
      1 12
      3 13
      2 14
student@okan-lab:~/lab$ grep WARN logs/system.log | cut -c27- | sort | uniq -c
     11 high memory usage
     17 temperature 71C
```

```
student@okan-lab:~/lab$ tail -n +2 data/students.csv | cut -d, -f3 | sort | uniq -c | sort -rn
     17 ECT
     11 ME
     10 CS
      2 EE
student@okan-lab:~/lab$ tail -n +2 data/students.csv | sort -t, -k5 -nr | head -n 3 | cut -d, -f2,3,5
Onur Dogan,ME,100
Elif Yilmaz,ECT,99
Emre Dogan,ME,98
```

Lab: Yönlendirme ve grep (A ve B bölümleri)

A1 data/ dizininin dosya listesini list.txt'ye kaydedin, sonra docs/ listesini ekleyin. Kaç satır?

A2 ls data nosuch komutunu, liste ok.txt'ye, hata err.txt'ye gidecek şekilde çalıştırın.

A3 logs/system.log'un yalnızca satır sayısını yazdırın — dosya adı olmadan.

A5 find . -name "*.md" okuyamadığınız bir klasör için yakınıyor. Yalnızca hataları gizleyin.

B2 system.log'un INFO OLMAYAN ilk 3 satırını satır numaralarıyla gösterin.

B3 access.log'daki kaç istek 404 durumunu aldı?

B4 Hangi öğrenciler EE bölümünde?

B5 10.0.5.38 adresinden kaç istek geldi? (Noktalara dikkat!)

Lab: Tek Satırlık Boru Hatları (C ve D bölümleri)

C1 Her bölümde kaç öğrenci var?

C2 Final notuna göre ilk 3 öğrencinin adları ve final notları.

C4 Web sitesinin en çok istenen 3 sayfası.

C5 En çok 404 hatasını hangi sayfa üretti?

C6 system.log her saatte kaç ERROR satırı içeriyor?

D1 students.csv'de hangi öğrenci adları birden çok kez geçiyor?

D2 Tüm 500 durumlu satırları errors_500.txt'ye kaydedin VE kaç tane olduklarını yazdırın — tek satırda.

D4 Siteyi kaç farklı IP adresi ziyaret etti? En az isteği hangisi yaptı?

Herkesin Bir Kez Yaptığı Altı Hata

```
student@okan-lab:~/lab$ cp data/cities.txt c.txt
student@okan-lab:~/lab$ sort c.txt > c.txt
student@okan-lab:~/lab$ wc -l c.txt
0 c.txt
student@okan-lab:~/lab$ grep user login logs/system.log
grep: login: No such file or directory
logs/system.log:2026-10-12 09:44:00 INFO  user login
...
student@okan-lab:~/lab$ ls data > wc
student@okan-lab:~/lab$ ls
README.txt  c.txt  docs  photos    scripts  wc
backup      data   logs  projects  src
student@okan-lab:~/lab$ sort -t, -k5 -n data/students.csv |
head -n 2
id,name,department,midterm,final
2026032,Onur Dogan,ECT,41,35
```

sort f > f

Kabuk f'yi sort okumadan ÖNCE boşaltır: 0 satır. Yeni dosyaya yazın, sonra mv.

grep user login f

Tırnaksız: grep "user"ı login adlı bir dosyada arar. "user login" yazın.

ls data > wc

> | değildir. Bu, wc adlı bir dosya oluşturur. ls data | wc -l yazın.

Başlık satırı

"id,name,..." -n ile 0 gibi sıralanır. Atlayın: tail -n +2.

sort'suz uniq

Yalnızca bitişik tekrarları sayar. sort | uniq -c kullanın.

2>&1 > f

Sıra soldan sağadır: hata ekranda kalır. > f 2>&1 yazın.

KISA SINAV

Kendinizi Sınayın

1. kmt > out.txt 2>&1 ile kmt 2>&1 > out.txt arasındaki fark nedir?

2. Antalya 6 kez geçtiği hâlde uniq -c data/cities.txt neden "1 Antalya" bildirir? Düzeltin.

3. logs/access.log'da en çok istek yapan 3 IP adresini yazdıran bir boru hattı yazın.

4. Kabuk grep ERROR logs/system.log | wc -l komutunu çalıştırmak için hangi sistem çağrılarını kullanır?

12. Hafta: Akılda Kalması Gerekenler

- **Standart akışlar:** her süreç fd 0 (stdin), 1 (stdout), 2 (stderr) ile başlar — 2–3. haftalarda izlediğimiz aynı write(1, …).
- **Yönlendirme:** > >> < 2> 2>&1 &> /dev/null. Kabuk dosyayı açar ve fork() ile exec() arasında dup2() kullanır; program bunu hiç bilmez.
- **Sıra:** yönlendirmeler soldan sağa uygulanır: > f 2>&1 iki akışı da yakalar, 2>&1 > f yakalamaz.
- **Borular:** kmt1 | kmt2 iki süreci eşzamanlı çalıştırır; aralarında bir çekirdek tamponu vardır (sınırlı tampon, 5. hafta).
- **Filtreler:** grep (-i -n -v -c -w -r, ^ $. [ ] *), sort (-n -r -t -k), uniq (-c -d, sort'tan sonra), cut, wc, tr, head, tail, tee.
- **find + xargs:** ağaçta ad, tür, boyut ve yaşa göre arama; boşluklu adlar için -print0 | xargs -0.
- **Yöntem:** çıkar → sort → uniq -c → sort -rn → head; her seferinde bir aşama.

GELECEK HAFTA

Süreçler ve İş Denetimi (13. hafta)

13. haftanın konuları:

- ps, pstree, top: süreçleri, PID'leri ve durumları izlemek (3. hafta)
- Ön plan ve arka plan: &, jobs, fg, bg, Ctrl+Z, Ctrl+C
- Sinyaller: kill, SIGTERM ile SIGKILL — ve yes | head neden durur?
- nice ve renice: süreç önceliği (5. hafta)
- Kendi gözünüzle görebileceğiniz bir zombi süreç

SORULAR?

Tartışma

Dersten sonra sorularınız olursa dersin Telegram kanalında sorun — yanıttan herkes yararlanır.

Dr. Öğr. Üyesi Hasan Oğuz